

Assignment 3 : Cache Demonstrator

Name: Sayalee Khedekar

MIS: 612303095

DIV:1

AIM :

To demonstrate and analyze the associativity and performance results of cache memory using Set Associative Cache and Direct Mapped Cache.

INTRODUCTION :

Modern processors utilize cache memory to minimize the performance gap between the CPU and main memory. A well-designed cache system can significantly improve overall system performance by reducing the average memory access time.

This report focuses on two fundamental cache mapping techniques and their impact on system performance:

- **Set-Associative Cache Mapping (T1):** Demonstrates how set associativity influences the placement and replacement of cache blocks, offering a balance between direct-mapped and fully associative caches.
- **Direct Mapped Cache (T4):** Demonstrates a simpler cache mapping strategy where each block maps to exactly one location in the cache, emphasizing speed and implementation simplicity.

Directly Mapped Cache (T4):

Direct mapping is a technique used to map blocks from main memory to cache lines in a one-to-one manner. In this approach, each memory block is assigned to exactly one specific cache line, determined by the formula:

Cache Line Number = (Main Memory Block Number) mod (Number of Cache Lines)

Because multiple memory blocks can map to the same cache line, a tag is used to identify which specific block is currently stored in that line. This ensures that even if different blocks map to the same location at different times, the cache can correctly determine if a requested block is present.

Working :

In direct mapping, the cache is divided into blocks, and each block in main memory has a fixed location in the cache. When a memory request is made, the cache controller checks if the block is present by comparing the tag. If the tag matches, it's a cache hit, and the data is retrieved from the cache. If not, it's a cache miss, and the block is fetched from main memory and placed in the cache, replacing the existing block. This method is simple but can cause frequent replacements, leading to higher cache misses.

Block Placement for various cache schemes [Main Menu](#)

Direct Mapped Cache ☒ Cache Size Block Size Associativity

Set Associative Cache ☐

OUTPUT

TAG									INDEX						OFFSET	
0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	1	0

Value in Decimal Format = 3

BLOCK	Value
BLOCK 0	0, 64, 128, 192, 256, 320, ...
BLOCK 1	1, 65, 129, 193, 257, 321, ...
BLOCK 2	2, 66, 130, 194, 258, 322, ...
BLOCK 3	3, 67, 131, 195, 259, 323, ...
BLOCK 4	4, 68, 132, 196, 260, 324, ...
BLOCK 5	5, 69, 133, 197, 261, 325, ...
BLOCK 6	6, 70, 134, 198, 262, 326, ...
BLOCK 7	7, 71, 135, 199, 263, 327, ...
BLOCK 8	8, 72, 136, 200, 264, 328, ...

☐ Binary
☒ Decimal

Maximum Decimal Number: 64K

- Cache Size : 128B
- Block Size : 2B
- Number of bits for main memory addresses = 16

Memory addresses are broken down into:

- TAG bits (leftmost bits)
- INDEX bits (middle)
- OFFSET bits (rightmost)

The address breakdown would be:

- Block/Word offset (w): 1 bit
 - Since block size is 2B, you need $2^1 = 2$ positions to address bytes within a block
 - Therefore $w = 1$ bit
- Cache line selector/Index (r): 6 bits
 - Number of cache lines = $\text{Cache Size} \div \text{Block Size} = 128B \div 2B = 64$ lines
 - To address 64 lines, you need $2^6 = 64$, so $r = 6$ bits
- Tag: 9 bits
 - Total address bits = 16
 - Tag bits = $16 - 6 - 1 = 9$ bits

So the 16-bit address is decoded as: [9 bits for TAG][6 bits for INDEX][1 bit for OFFSET]

The blue highlight on "BLOCK 3" indicates that memory address 67 would be placed in block 3 of the cache. (67 in binary is 1000011)

Advantages of Direct Mapping:

1. **Simplicity:** The mapping process is straightforward and easy to implement.
2. **Low Hardware Requirements:** It requires less complex hardware compared to other mapping techniques.
3. **Faster Access:** Memory accesses can be quicker since there's a one-to-one mapping between memory blocks and cache lines.

Disadvantages of Direct Mapping:

1. **High Cache Miss Rate:** Multiple memory blocks mapping to the same cache line can cause frequent replacements, leading to higher cache misses.
2. **Inefficient Use of Cache:** If several blocks frequently map to the same line, the cache may not be fully utilized, affecting performance.
3. **Limited Flexibility:** The fixed mapping reduces flexibility, making it harder to optimize cache usage for different workloads.

Set Associative Cache (T4):

Set-Associative Mapping is a technique that maps memory blocks to a set of cache lines rather than a single cache line. In this approach, each memory block is assigned to a specific set, and within that set, it can be placed in any of the cache lines. The set is determined by the formula:

$$\text{Cache Set Number} = (\text{Main Memory Block Number}) \bmod (\text{Number of Sets})$$

Each set contains multiple cache lines, and when a memory block maps to a set, it can be stored in any of the available lines within that set. A tag is used to identify the block stored in each line within the set.

Working:

In set-associative mapping, the cache is divided into sets, each containing multiple lines. When a memory request is made, the cache controller checks the set that corresponds to the memory block. It then compares the tags in the cache lines of that set. If the tag matches, it's a cache hit, and the data is retrieved. If no match is found, it's a cache miss, and the block is fetched from main memory, replacing one of the blocks in the set. This method reduces the frequency of replacements compared to direct mapping and helps lower cache misses. However, it is more complex and requires more hardware than direct mapping.

Block Placement for various cache schemes [Main Menu](#)

☐ Direct Mapped Cache
 Cache Size
 Block Size
 Associativity

☒ Set Associative Cache

OUTPUT

TAG										INDEX				OFFSET
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

Value in Decimal Format = 1

SET	0	1	2	3	4	5	Addresses
SET 0							0, 16, 32, 48, 64, 80, ...
SET 1							1, 17, 33, 49, 65, 81, ...
SET 2							2, 18, 34, 50, 66, 82, ...
SET 3							3, 19, 35, 51, 67, 83, ...
SET 4							4, 20, 36, 52, 68, 84, ...
SET 5							5, 21, 37, 53, 69, 85, ...

☐ Binary
 ☒ Decimal

MAP

Maximum Decimal Number: 64K

- Cache Size : 128B
- Block Size : 2B
- Associativity: 4 way
- Number of bits for main memory addresses = 16

Memory addresses are broken down into:

- TAG bits (leftmost bits)
- INDEX bits (middle)
- OFFSET bits (rightmost)

For a 4-way associative cache, the address breakdown would change:

- TAG:** 10 bits
 - Bits: 0000000000 (leftmost section)
 - Purpose: Identifies which memory block is stored in the cache line
- INDEX:** 5 bits
 - Bits: 00001 (middle section)
 - Purpose: Determines which cache set to use
 - With 4-way associativity, we'd have:
 - $128B \div (2B \times 4) = 16$ sets (need 4 bits to address)
 - The value 00001 = 1 in decimal, meaning this address maps to Set 1
- OFFSET:** 1 bit
 - Bit: 0 (rightmost bit)
 - Purpose: Selects which byte within the 2-byte block

33 in binary is 0000000000100001

This would be divided into:

- TAG:** 10 bits = 0000000010
- INDEX:** 5 bits = 00001
- OFFSET:** 1 bit = 0

The Index value is 00001 = 1 in decimal, which means this address would map to Set 1 in the cache.

In a 4-way associative cache, Set 1 can hold 4 different blocks (with different tag values). Since the address 33 has a tag value of 0000000010, it would be stored in Set 1 alongside up to 3 other blocks with different tags.

The specific block within Set 1 would depend on the replacement policy (like LRU, FIFO, etc.) if the set is already full. If there's an empty position in Set 1, the block containing address 33 would be placed in that position.

Advantages of Set-Associative Mapping:

1. **Better Hit Rate:** Reduces conflict misses by allowing multiple cache lines per set.
2. **Flexibility:** More options for storing data, reducing cache misses.
3. **Balanced Performance:** Offers a good trade-off between complexity and performance.

Disadvantages:

1. **Increased Complexity:** Requires checking multiple locations per set.
2. **Slower Access:** Checking multiple slots can increase access time.
3. **Higher Cost:** Needs more hardware, like comparators, than direct-mapped caches.
4. **Diminishing Returns:** More associativity improves hits but increases cost with limited performance gain.

Conclusion:

Direct-mapped cache uses one cache line per memory block, resulting in faster access but lower hit rates and simpler implementation, making it cheaper. In contrast, set-associative cache allows multiple cache lines per memory block, improving hit rates but with slower access, higher complexity, and greater cost.